

Project 2: Neo4j TestPilot - Fundamentals & Deep Dive

This document provides a deep dive into the core technologies, protocols, and architectural concepts that power the Neo4j TestPilot project.

Knowledge Graphs & Neo4j

1. What is a Knowledge Graph?

A knowledge graph is a way of representing data as a network of entities and the relationships between them. Instead of storing data in rigid tables with rows and columns (like a relational database), it uses a flexible graph structure consisting of:

- **Nodes:** Represent entities (e.g., a `Class`, a `Method`, a `GitVersion`).
- **Relationships (or Edges):** Represent the connections between entities (e.g., a `Class` `[:DEFINES_METHOD]->` a `Method`).
- **Properties:** Key-value pairs that store data on nodes and relationships (e.g., a `Class` node with a property `name: "GenericTestMethod"`).

2. Why Neo4j over a Relational (SQL) Database?

For this project, a graph database like Neo4j was a fundamentally better choice than a SQL database for several reasons:

- **Natural Relationship Modeling:** The core of the project is understanding the complex, interconnected relationships within a codebase (inheritance, dependencies, method calls). A graph model represents these connections directly and intuitively. In SQL, this would require numerous complex and inefficient JOIN tables.

- **Query Performance for Path Traversal:** Answering questions like “Find all classes that inherit from this base class, up to 3 levels deep” is extremely fast in Neo4j because it involves traversing direct pointers between nodes. In SQL, this would require slow, recursive queries.
- **Schema Flexibility:** As the understanding of the codebase evolves, new types of nodes and relationships can be added to the graph without requiring disruptive schema migrations that are common in SQL databases.
- **Powerful Visualization:** The ability to visually explore the graph in the Neo4j Browser is invaluable for debugging and understanding the structure of the legacy code.

3. The Cypher Query Language

Cypher is Neo4j’s declarative query language, designed to be intuitive and human-readable. It uses ASCII-art syntax to represent graph patterns.

Example Query: Find all methods defined by the `GenericTestMethod` class.

```
MATCH (c:Class {name: "GenericTestMethod"})-[:DEFINES_METHOD]->(m:Method)
RETURN m.name, m.signature
```

- `(c:Class {name: ...})` matches a **node** aliased as `c`, with the label `Class` and a specific `name` property.
 - `-[:DEFINES_METHOD]->` matches a **relationship** with the type `DEFINES_METHOD` pointing from `c` to `m`.
 - `(m:Method)` matches the connected **node** with the label `Method`.
-

Core Technologies & Protocols

1. Model Context Protocol (MCP)

What is it?

MCP is an open standard that allows Large Language Models (LLMs) like GitHub Copilot to interact with external tools and data sources. It acts as a universal translator, defining a common language for an LLM to discover what tools are available, understand their capabilities, and invoke them.

How does it work?

1. **Discovery:** The LLM asks the MCP server (in this project, the `GenAI Toolbox`) for a list of available tools and their specifications (defined in the `neo4j_tools.yaml` file).
2. **Invocation:** When a user's prompt requires an external tool, the LLM sends a request to the MCP server to execute a specific tool with certain parameters (e.g., `search_class_by_name(class_name='GenericTestMethod')`).
3. **Execution:** The MCP server receives the request, executes the corresponding code (e.g., runs a Cypher query against Neo4j), and gets a result.
4. **Response:** The server sends the result back to the LLM in a structured format. The LLM then uses this result as context to generate its final response (e.g., the Java code).

Why was it chosen?

MCP is the key that unlocks the knowledge graph for the LLM. Without it, GitHub Copilot would have no way of knowing the Neo4j database exists or how to query it. MCP provides the essential bridge, transforming the LLM from a general-purpose code generator into a domain-specific expert that can leverage the structured knowledge of the entire codebase.

2. Tree-sitter Parser

What is it?

Tree-sitter is a modern parser generator tool. It takes a formal grammar for a programming language (like Java) and generates a library that can parse source code into a concrete syntax tree (CST). It is designed to be fast, robust, and incremental.

How does it work?

Unlike simple regex-based parsers, Tree-sitter builds a complete, structured understanding of the code. It recognizes every element—classes, methods, variables, loops, etc.—and their hierarchical relationships. Its key advantage is **error tolerance**. If it encounters a syntax error, it doesn't give up; it enters an "error mode" and attempts to synchronize its state, allowing it to continue parsing the rest of the file.

Why was it chosen?

Legacy codebases are rarely perfect. An initial attempt to parse the 481 Java classes with a simpler parser failed on 12% of the files. Tree-sitter was chosen because its robustness and 100% success rate were non-negotiable for building a complete and accurate knowledge graph. Its speed and ability to generate a detailed syntax tree were also critical for extracting all the necessary information (methods, fields, dependencies).

3. LLM for Semantic Analysis

What is it?

This refers to the technique of using a general-purpose LLM (like GPT-4) to analyze the `git diff` between two versions of a file to identify "semantic" changes, not just textual ones.

How does it work?

The process involves:

1. Generating a textual `diff` of a code file between two commits.

2. Feeding this diff into an LLM with a carefully crafted prompt, asking it to identify specific types of changes, such as:
 - Changes to method signatures (a breaking change).
 - Removal of import statements (a potential dependency issue).
 - Changes in variable types.
3. The LLM, with its understanding of programming language syntax and structure, can interpret the diff and provide a structured JSON output detailing these semantic changes.

Why was this approach used?

A simple textual diff can tell you *what* lines changed, but not *what the change means*. By leveraging an LLM, the system can automatically flag potentially breaking changes that would otherwise require a manual code review to detect. This adds a layer of intelligent, automated quality assurance to the version comparison process.